

Università degli Studi di Milano

Corso di Laurea in sicurezza dei sistemi e delle reti informatiche

Esame di Programmazione Web e Mobile – A.A. 2025/2026



FAST FOOD

Progettazione e Sviluppo di un Sistema RESTful per la Gestione di Ordini

Candidato: Davide Colombo

Matricola: 61779A

Anno Accademico 2025/2026

1. Introduzione e Analisi dei Requisiti
 - 1.1 Obiettivi del Progetto
 - 1.2 I Quattro Macro-Scenari Operativi
2. Architettura del Sistema e Stack Tecnologico
 - 2.1 Livello di Presentazione (Sviluppo Frontend)
 - 2.2 Livello di Business Logic e API (Sviluppo Backend)
 - 2.3 Analisi e Giustificazione delle Scelte Implementative
3. Modellazione dei Dati (Database NoSQL)
4. Analisi Funzionale e Moduli dell'Applicativo
 - 4.1 Modulo di Autenticazione e Gestione RBAC
 - 4.2 Motore di Ricerca Asincrono e Filtraggio
 - 4.3 Gestione dello Stato del Carrello e Checkout
 - 4.4 Dashboard Ristoratore: Gestione Dinamica del Menu e Statistiche
 - 4.5 Macchina a Stati per il Ciclo di Vita dell'Ordine
5. Conclusioni

1. Introduzione e Analisi dei Requisiti

1.1 Obiettivi del Progetto

Il presente documento illustra l'analisi funzionale, la progettazione architeturale e lo sviluppo pratico dell'applicativo web denominato "Fast Food". Il sistema è stato concepito per rispondere alla specifica di progetto proposta in sede d'esame, la quale richiedeva la realizzazione di una piattaforma completa per la gestione di ordini online per catene di ristorazione. L'obiettivo primario è stato quello di fornire un'infrastruttura digitale robusta, di tipo two-sided, capace di far interagire due attori principali (Clienti e Ristoratori) in un ambiente transazionale sicuro, fluido e asincrono.

1.2 I Quattro Macro-Scenari Operativi

Seguendo le specifiche richieste nel documento del progetto, ho strutturato l'applicativo per coprire i quattro scenari principali:

1. *Gestione del Profilo Utente*: l'applicativo prevede un sistema di registrazione e profilazione con separazione dei privilegi. Gli utenti definiscono la propria natura (Cliente o Ristoratore) fin dal primo accesso, permettendo al sistema di instradare la navigazione verso interfacce e logiche di business completamente differenti.
2. *Gestione del Ristorante*: è stato sviluppato un pannello amministrativo dedicato ai proprietari dei locali. Questo ambiente permette di configurare l'identità commerciale del ristorante e di popolare dinamicamente il catalogo dei prodotti (menu), sia attingendo da un dataset centralizzato, sia inserendo piatti personalizzati.
3. *Gestione degli Ordini*: per il consumatore finale è stata implementata un'interfaccia di esplorazione avanzata. Il cliente può interrogare il database alla ricerca di piatti e ristoranti, compilare un carrello virtuale multiprodotto e finalizzare l'ordine simulando un checkout di pagamento reale.
4. *Gestione delle Consegne*: il sistema modella il flusso logistico dell'ordine. I ristoratori ricevono le comande in tempo reale, potendo segnalare l'inizio della preparazione e la successiva evasione, mentre i clienti monitorano l'avanzamento fino alla conferma di ricezione.

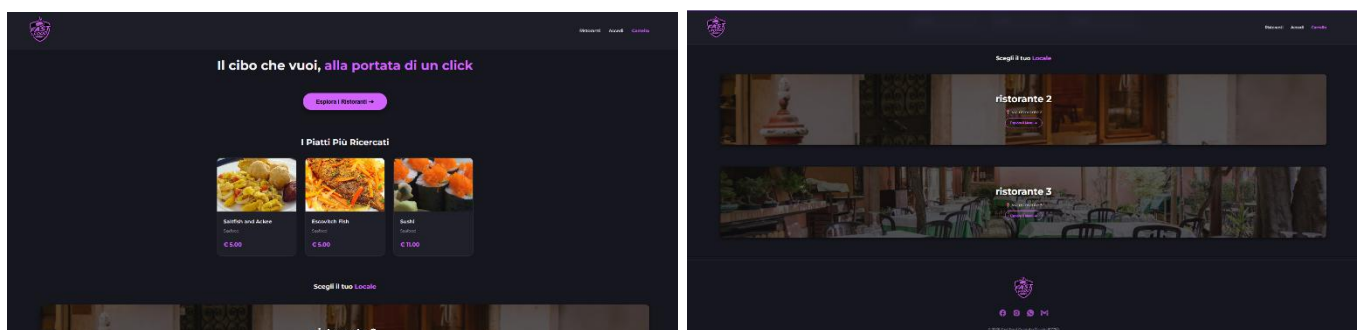


Figura 1-2: Interfaccia principale (Home Page) dell'applicativo "Fast Food"

2. Architettura del Sistema e Stack Tecnologico

Per l'applicativo ho scelto di seguire l'architettura Client-Server basata su API RESTful. Questo mi ha permesso di separare nettamente il livello di presentazione tra il livello di presentazione (Frontend) e il livello di persistenza e business logic (Backend).

2.1 Livello di Presentazione (Sviluppo Frontend)

Il client è stato realizzato rigorosamente in puro HTML5, CSS3 e Vanilla JavaScript. Ho deciso di non utilizzare framework esterni (come React o Vue) per lavorare direttamente sul codice nativo e gestire in autonomia la manipolazione del DOM.

Il lavoro lato Frontend si è concentrato su tre pilastri tecnologici:

- *Comunicazione Asincrona (Fetch API)*: tutte le interazioni con il database avvengono "dietro le quinte" tramite l'oggetto Promise nativo di JavaScript. L'utilizzo massiccio della sintassi `async/await` permette di interrogare il backend, filtrare dati e inviare ordini senza mai dover ricaricare la pagina web, offrendo una fluidità di navigazione tipica di una Single Page Application (SPA).
- *Gestione dello Stato Globale (Web Storage API)*: i dati necessari al mantenimento della sessione (come il Token JWT, il ruolo dell'utente e gli array strutturali del carrello acquisti) vengono persistiti nella RAM del browser tramite l'uso del `localStorage`. Questo permette all'interfaccia di mantenere lo stato dell'applicazione coerente anche a fronte di ricaricamenti accidentali della finestra.
- *Per migliorare la User Experience*, si è evitato l'uso dei classici e bloccanti `alert()` di sistema. È stato invece ingegnerizzato un motore di notifiche "Toast" proprietario: funzioni JavaScript iniettano dinamicamente nel DOM dei pop-up non intrusivi, che informano l'utente sull'esito delle operazioni (successo o errore) autogestendo le proprie animazioni CSS e i timeout di chiusura.

2.2 Livello di Business Logic e API (Sviluppo Backend)

Il server è stato sviluppato in ambiente runtime Node.js sfruttando il framework Express.js. Il backend non genera pagine HTML (assenza di templating engine), ma agisce esclusivamente come strato di servizi RESTful, rispondendo tramite payload in formato JSON. Le principali implementazioni lato server includono:

- *Routing Modulare*: L'infrastruttura di rete è stata suddivisa logicamente in router distinti per dominio di competenza (es. gestione auth, ristoranti, ordini), mappando i verbi HTTP (GET, POST, PUT, DELETE) sulle operazioni CRUD del database.
- *Sicurezza e Middleware (JWT)*: L'autorizzazione delle rotte protette è delegata a middleware personalizzati. Ogni richiesta sensibile viene intercettata dal server, che verifica la decodifica e la validità temporale del token JWT presente nell'intestazione HTTP (Authorization: Bearer). In assenza di autorizzazione, il server blocca preventivamente l'esecuzione restituendo uno status code 401 Unauthorized.

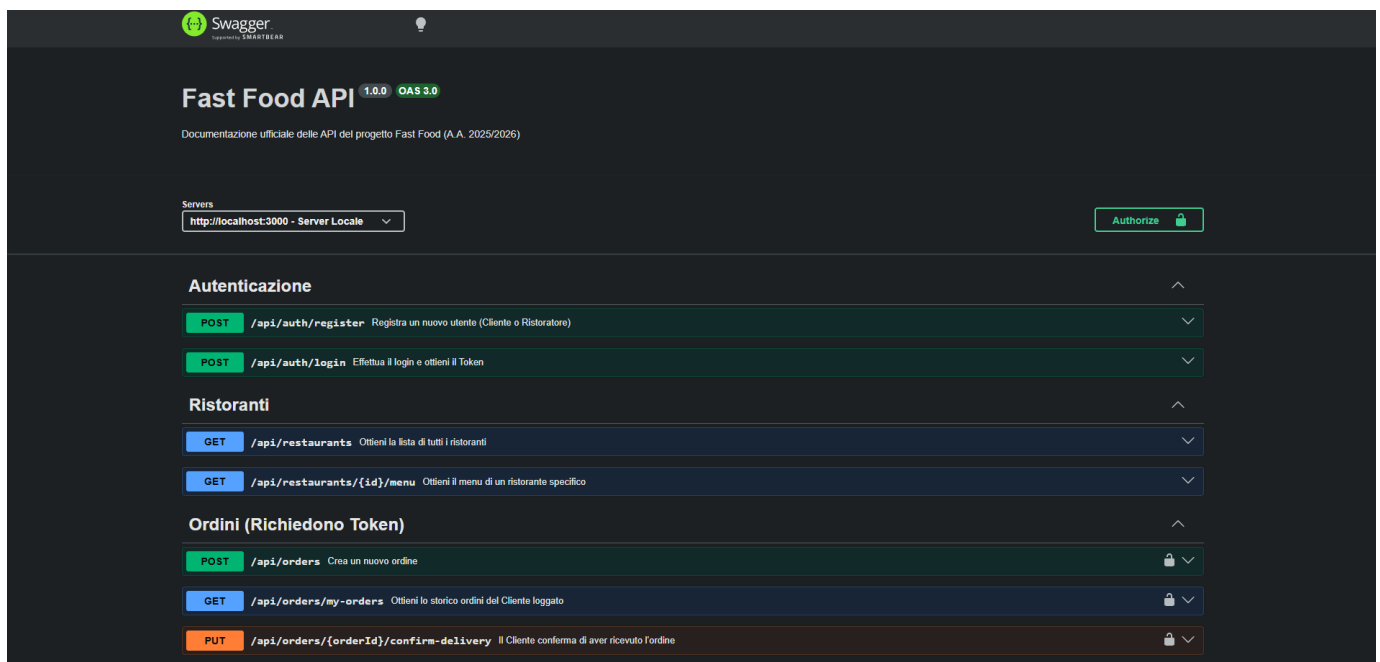


Figura 3: Interfaccia Swagger per la documentazione e il collaudo interattivo delle API RESTful.

2.3 Analisi e Giustificazione delle Scelte Implementative

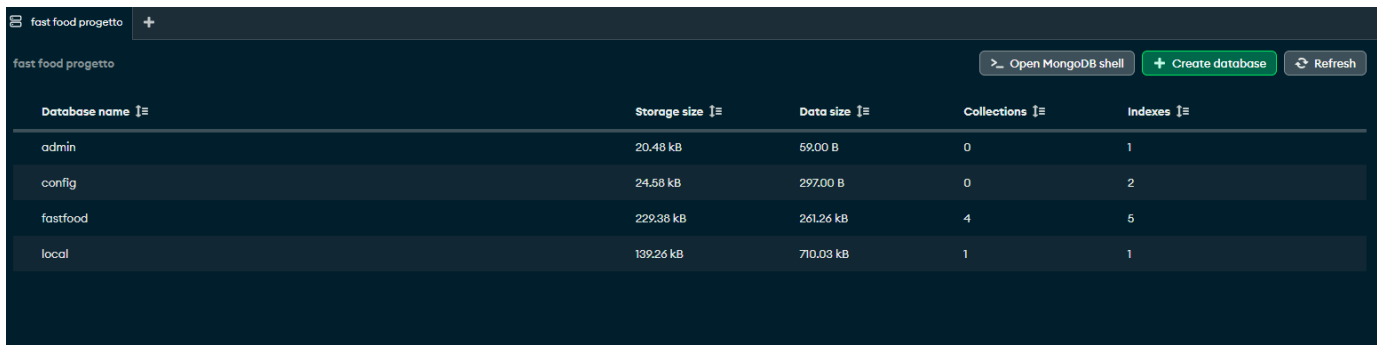
Come richiesto dalle specifiche, ho valutato diverse alternative prima di implementare le funzionalità. Per ottimizzare le prestazioni e la sicurezza ho preso queste decisioni principali:

- *Gestione Immagini tramite URL*: ho preferito non gestire l'upload fisico delle immagini nel database per non appesantire MongoDB. Ho quindi inserito un campo dove i ristoratori incollano direttamente l'URL pubblico della fotografia. Questa architettura impedisce l'esplosione dimensionale dei documenti del database, mantiene i tempi di query estremamente rapidi e sfrutta la rete esterna per il rendering grafico, alleggerendo il carico del server Node.js.
- *Simulazione Transazionale (Interactive UX)*: per soddisfare il requisito del pagamento in fase di checkout, senza però violare le basilari norme di sicurezza informatica o le direttive PCI-DSS (evitando il salvataggio in chiaro di dati sensibili di carte di credito), si è optato per una soluzione di User Experience avanzata lato client. Una carta di credito grafica processa visivamente i dati in tempo reale durante la digitazione, sfruttando specifici Event Listeners sugli input form. Ciò offre grande realismo e interattività all'utente, pur mantenendo il backend completamente isolato da informazioni finanziarie critiche.
- *Esclusione di Automatismi Temporalì (CRON-Jobs)*: per gestire le consegne, ho evitato di creare script temporali per l'avanzamento automatico degli ordini. Ho ritenuto molto più realistico e coerente con le richieste che il ristoratore confermasse manualmente la fine della preparazione tramite gli appositi tasti. Questo approccio rispecchia fedelmente le dinamiche asincrone del mondo reale, dove i tempi di una cucina non sono mai perfettamente deterministici e richiedono un feedback umano diretto.

3. Modellazione dei Dati (Database NoSQL)

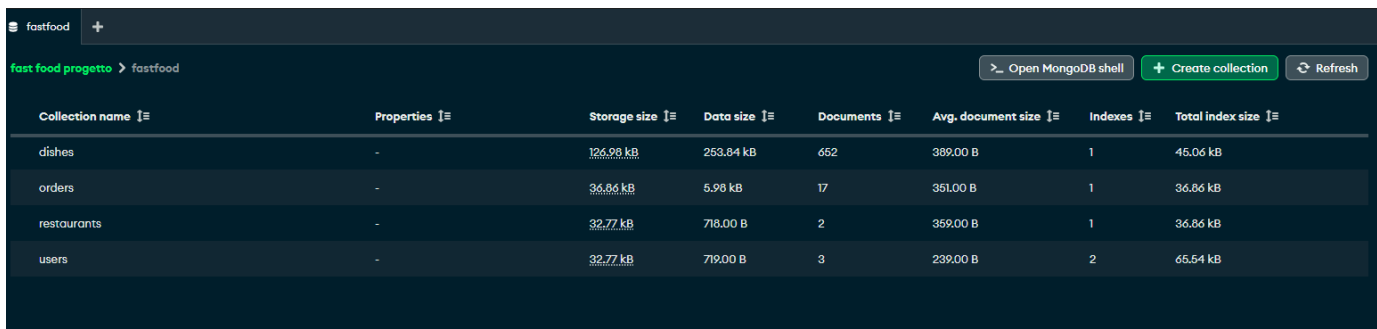
L'infrastruttura dati risiede su MongoDB, DBMS non relazionale scelto per l'elevata flessibilità offerta dai documenti BSON. Sono state modellate quattro collezioni principali, interconnesse logicamente:

- *Users*: gestisce le identità digitali, contenendo le credenziali criptate e gli attributi di autorizzazione (Ruolo).
- *Restaurants*: modella l'entità commerciale e funge da riferimento topologico per le ricerche dei clienti.
- *Dishes*: contiene il catalogo prodotti. Ogni documento preserva gli attributi del piatto (nome, prezzo, ingredienti) e include una chiave esterna logica (l'ObjectId del ristorante proprietario), simulando un legame uno-a-molti, vitale per le interrogazioni aggregate.
- *Orders*: registra le transazioni finanziarie e logistiche. I documenti catturano uno snapshot cristallizzato del carrello al momento dell'acquisto (prevenendo incongruenze storiche se i prezzi dei piatti dovessero variare in futuro) e tracciano l'evoluzione logistica della consegna.



The screenshot shows the MongoDB Atlas interface for a database named 'fast food progetto'. It displays a table with database-level statistics:

Database name	Storage size	Data size	Collections	Indexes
admin	20.48 kB	59.00 B	0	1
config	24.58 kB	297.00 B	0	2
fastfood	229.38 kB	261.26 kB	4	5
local	139.26 kB	710.03 kB	1	1



The screenshot shows the MongoDB Atlas interface for the 'fastfood' database, displaying a table with collection-level statistics:

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
dishes	-	126.98 kB	253.84 kB	652	389.00 B	1	45.06 kB
orders	-	36.86 kB	5.98 kB	17	351.00 B	1	36.86 kB
restaurants	-	32.77 kB	718.00 B	2	359.00 B	1	36.86 kB
users	-	32.77 kB	719.00 B	3	239.00 B	2	65.54 kB

Figura 4-5: Struttura documentale e popolamento delle collezioni nel DBMS MongoDB

4. Analisi Funzionale e Moduli dell'Applicativo

4.1 Modulo di Autenticazione e Gestione RBAC

Il sistema utilizza un *single-entry-point* per l'autenticazione. Il form di registrazione accorpa l'inserimento dei dati anagrafici e la fondamentale scelta del ruolo operativo. Il server, a seguito della convalida formale e dell'hashing della password, genera un Token crittografato. Il livello Frontend intercetta questo token e applica logiche di RBAC (*Role-Based Access Control*), rendendo visibili all'interno della NavBar e delle varie dashboard solamente le funzioni e i pulsanti pertinenti al ruolo dell'utente connesso.

The screenshot displays the authentication module of the FAST FOOD application. The interface is dark-themed with a purple accent color. At the top, there is a navigation bar with links: Home, Ristoranti, Accedi, and Carrello. The main content area is divided into two panels: 'benvenuto Registrati' on the left and 'Bentornato! Accedi' on the right. The 'Registrati' panel includes a dropdown menu for 'Chi sei?', input fields for 'Nome', 'Cognome', 'La tua Email', and 'Crea una Password', and a purple 'Crea Account' button. The 'Accedi' panel includes input fields for 'La tua Email' and 'Password', and a purple 'Accedi' button. The FAST FOOD logo is visible in the top left and bottom center.

Figura 6: Modulo di autenticazione integrato con selezione dinamica del ruolo utente.

4.2 Motore di Ricerca Asincrono e Filtraggio

La pagina di esplorazione costituisce il fulcro vitale dell'esperienza Cliente. Per soddisfare i requisiti, il backend impiega l'operatore \$regex di MongoDB per effettuare ricerche parziali e case-insensitive sui campi testuali. Lato frontend, è stata ingegnerizzata un'interfaccia a schede (Tabs) che permette di filtrare i ristoranti per località o di scendere nel dettaglio merceologico cercando specifici piatti tramite nome, tipologia e scaglioni di prezzo. I risultati vengono renderizzati in tempo reale sotto forma di Card dinamiche, generate tramite l'iterazione degli array JSON ricevuti dal server.

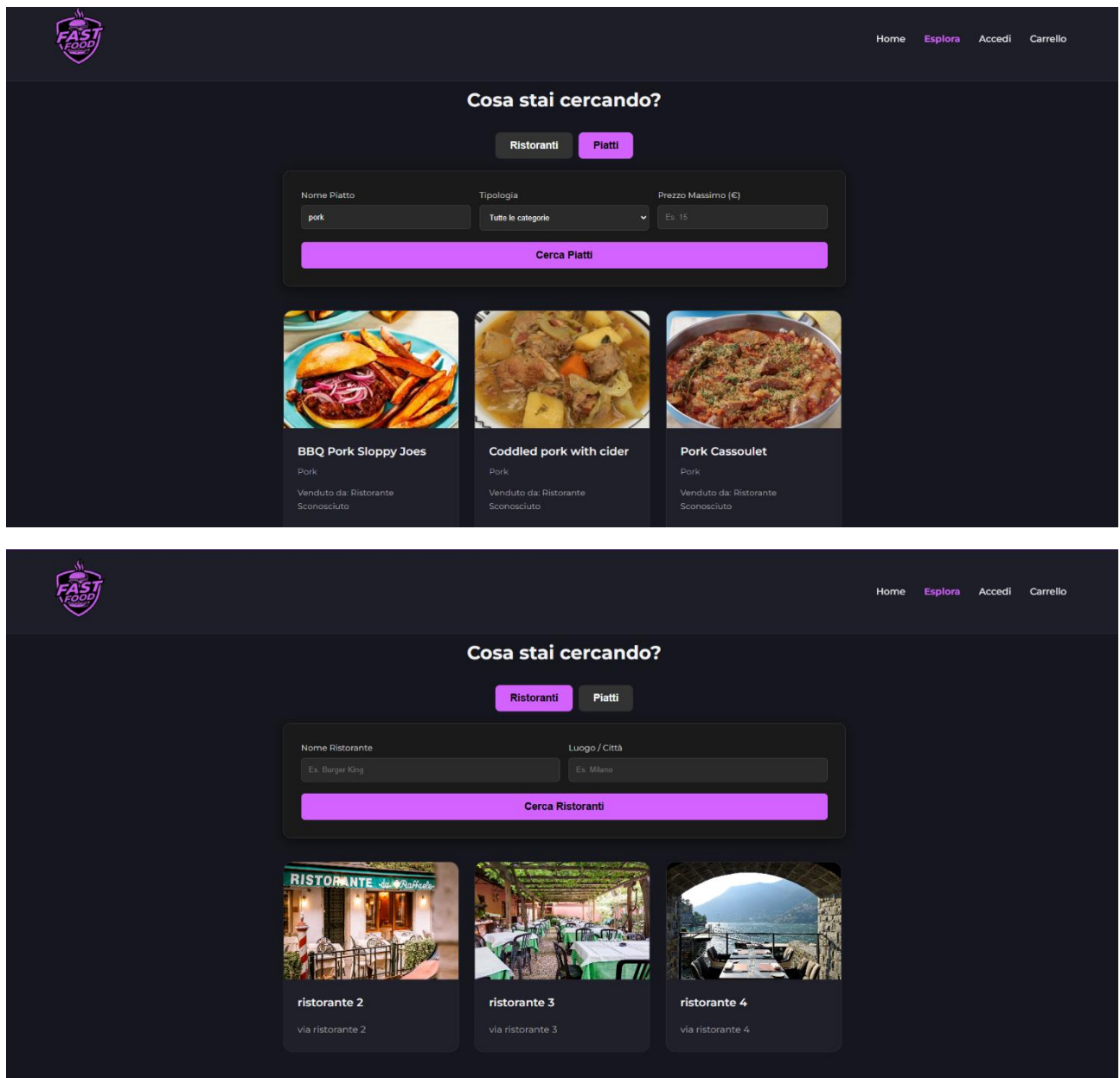


Figura 7-8: Dashboard di interrogazione asincrona del catalogo ristoranti e piatti

4.3 Gestione dello Stato del Carrello e Checkout

Il carrello è gestito come una struttura dati in-memory (RAM e LocalStorage). Il frontend intercetta i click dell'utente, estrae l'oggetto "piatto" e ne aggiorna l'array del carrello, ricalcolando le sommatorie matematiche del totale a ogni singola interazione. La fase di checkout culmina con la UI interattiva della carta di credito precedentemente giustificata; all'invio del modulo form, JavaScript serializza l'intero stato del carrello in un payload JSON e lo trasmette al server per l'effettivo salvataggio nel database e l'inizializzazione dell'ordine.

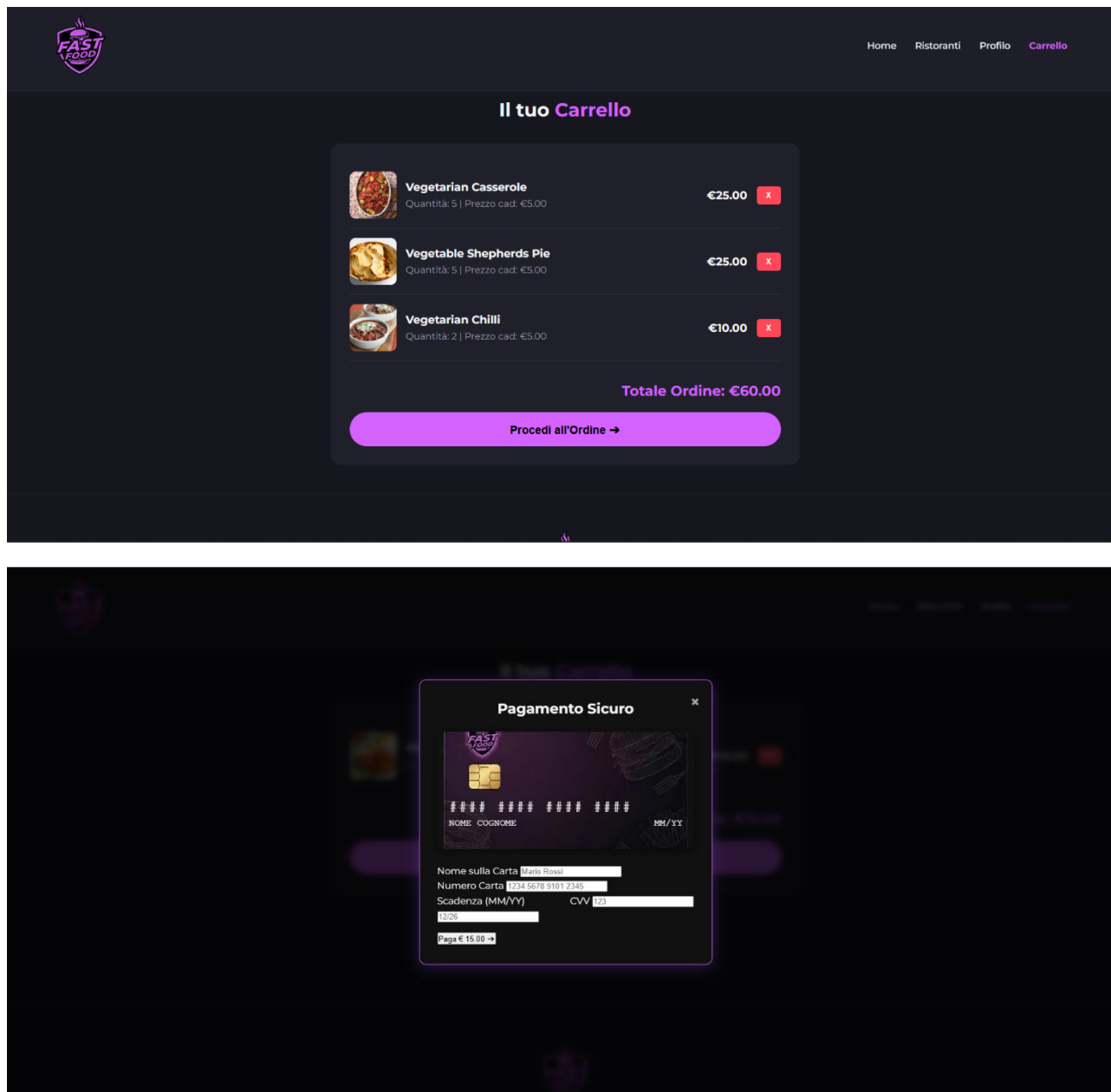


Figura 9-10: Modulo di composizione dell'ordine e simulazione interattiva del checkout

4.4 Dashboard Ristoratore: Gestione Dinamica del Menu e Statistiche

L'amministrazione della vetrina commerciale offre strumenti avanzati ai ristoratori. Per ottimizzare il data-entry, l'utente può popolare il menu attingendo rapidamente da un catalogo comune condiviso in formato JSON. Alternativamente, per massimizzare la personalizzazione, è presente un modulo di inserimento Custom. Il ristoratore può definire da zero un piatto inviando via POST parametri dettagliati, quali nome, tipologia, prezzo, la lista degli ingredienti e l'URL fotografico. Oltre alla gestione dei prodotti, la dashboard implementa un cruscotto analitico (Statistiche) in tempo reale. Il frontend interroga specifiche rotte API del backend che, tramite funzioni di aggregazione sulla collezione orders, restituiscono metriche fondamentali per il business: volume degli ordini completati, ammontare dei ricavi totali e carico di lavoro attuale (ordini in corso). Questo garantisce al gestore una visione analitica completa.

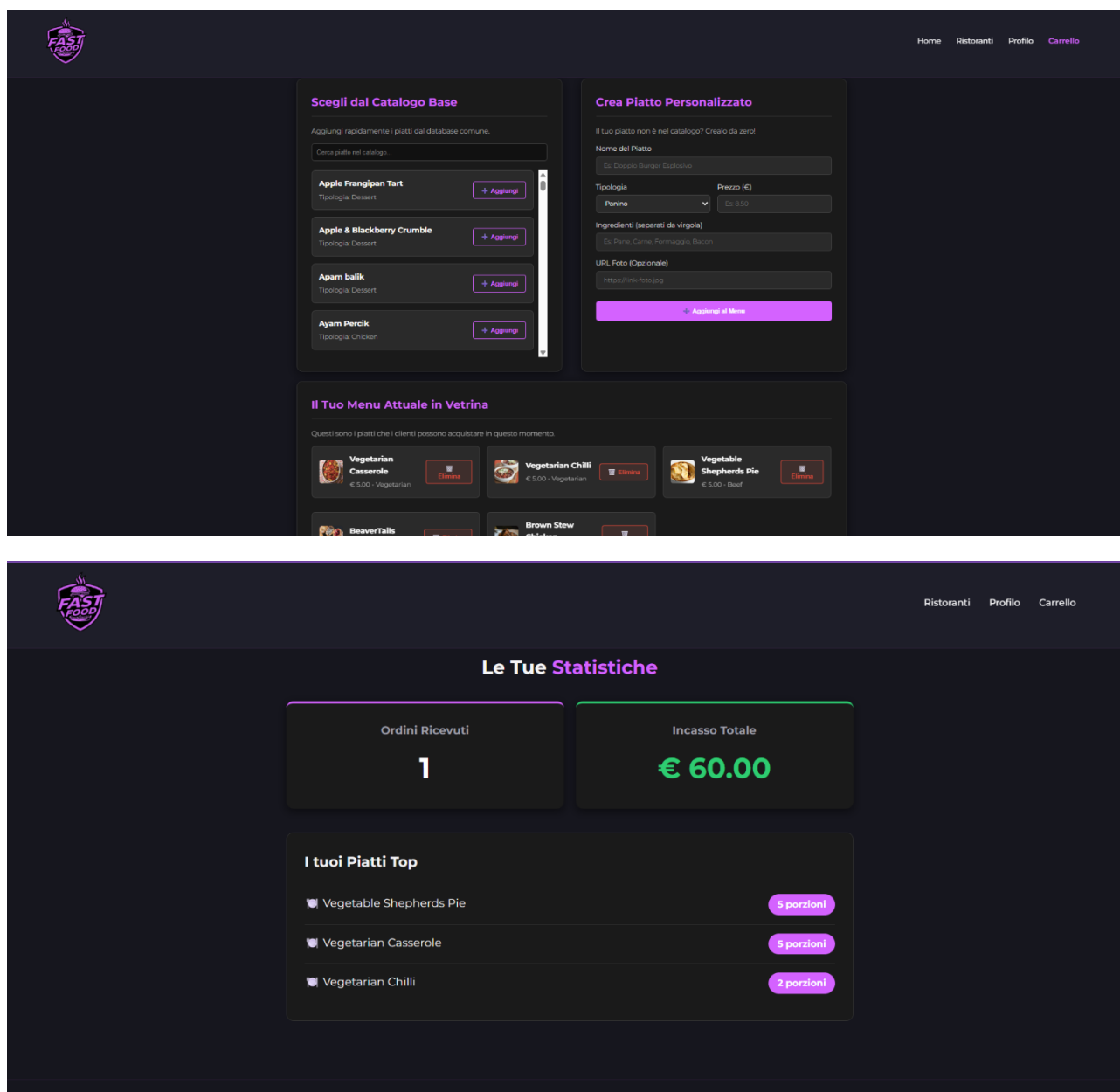


Figura 11-12: Pannello di controllo esclusivo per l'amministrazione del menu e l'analisi statistica delle performance.

4.5 Macchina a Stati per il Ciclo di Vita dell'Ordine

L'avanzamento logistico dell'ordine è stato implementato come una macchina a stati finiti, guidata dall'interazione umana diretta. Il Ristoratore, tramite la propria dashboard "Gestione Cucina", interroga il server per estrarre gli ordini a lui destinati. Tramite chiamate di rete PUT, egli è in grado di modificare la proprietà status del documento MongoDB, facendolo transitare verso lo stato "In preparazione" o "Consegnato". Il Cliente, dalla propria pagina personale, visualizza in tempo reale questi aggiornamenti, partecipando attivamente alla chiusura del ciclo logistico tramite il pulsante di conferma ricezione.

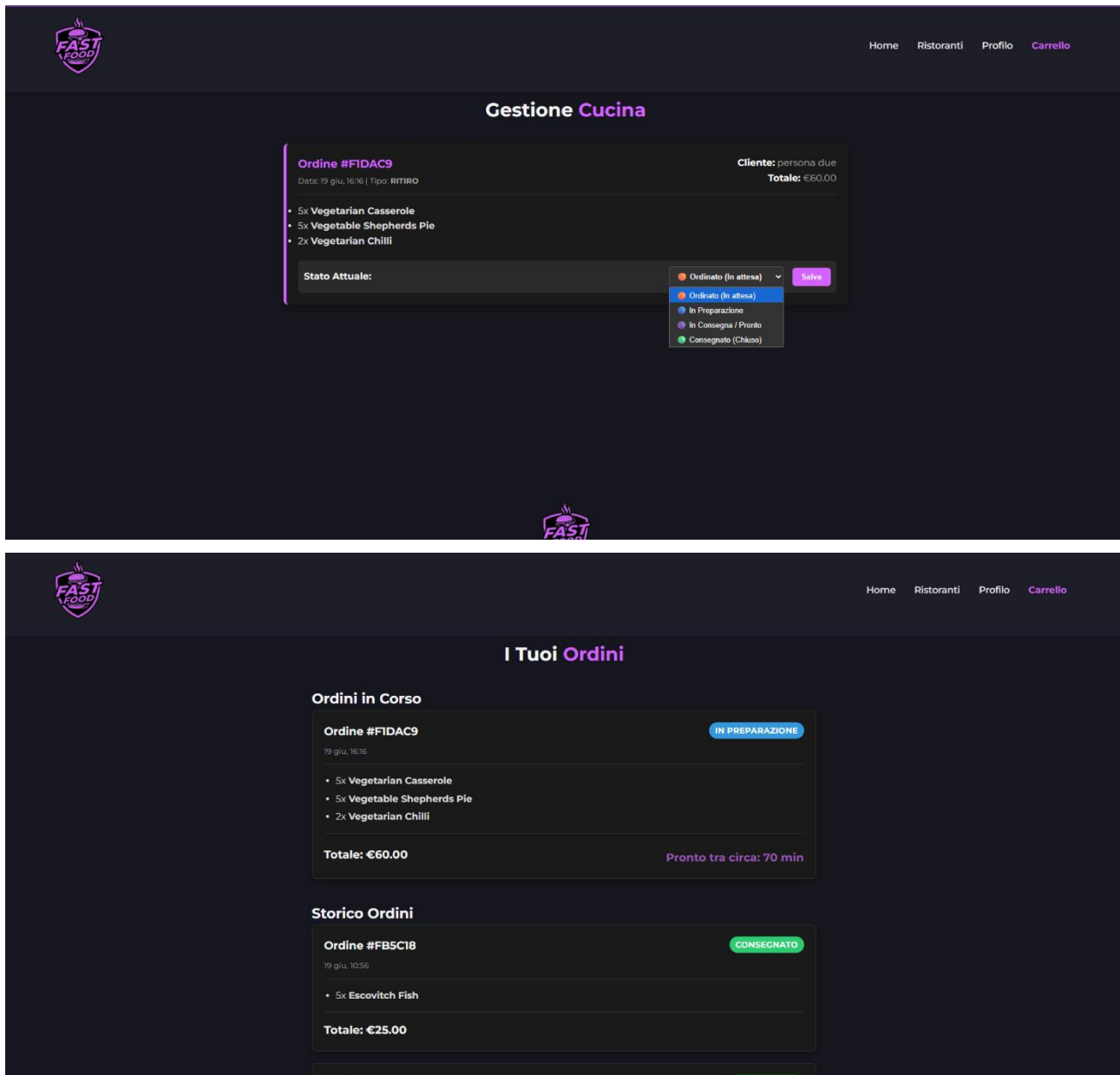


Figura 13-14: Interfaccia asincrona per il monitoraggio e la mutazione logica dello stato dell'ordine.

5. Conclusioni

L'architettura progettata e implementata soddisfa integralmente i requisiti funzionali e non funzionali descritti nella specifica d'esame. L'ingegnerizzazione di un Backend RESTful con Node.js e MongoDB ha garantito transazioni sicure e una gestione altamente efficiente dei dati. Parallelamente, lo sviluppo del Frontend in puro Vanilla JavaScript ha permesso di mettere in luce competenze fondamentali relative alla manipolazione del DOM, alla gestione dello stato applicativo e alle chiamate di rete asincrone. Il prodotto finale risulta essere un sistema stabile, robusto, aderente alle dinamiche reali del mercato del food delivery e perfettamente strutturato per scalare agevolmente in scenari futuri.